

Advanced Game-Theoretic Frameworks for Autonomous Agents in Turn-Based Hex-Grid Environments

The Evolution of Turn-Based Strategy Artificial Intelligence

The development of artificial intelligence for turn-based strategy games represents a frontier in game theory, computational complexity, and multi-agent system design. Unlike traditional perfect-information board games such as Chess or Go, modern digital strategy environments introduce profound complexities: massive combinatorial action spaces, heterogeneous entity types, multi-domain combat across ground and air environments, spatial reasoning on hexagonal topologies, and imperfect information due to mechanics such as fog of war and stealth. The challenge of engineering an optimal autonomous agent requires transcending standard search algorithms, demanding a sophisticated synthesis of hierarchical planning, probabilistic information state sampling, and hybrid neuro-symbolic cognitive architectures. Historically, the evaluation of such complex environments has drawn heavily from frameworks developed for titles like *Battle for Wesnoth* and *StarCraft*, where researchers have successfully implemented Monte Carlo Tree Search (MCTS) variants, behavior trees, and deep reinforcement learning to navigate the intractable state spaces.¹ The environment under analysis in this report operates on a discrete hexagonal grid, featuring complex unit interactions, economy management, and strict base destruction mechanics. Compounding the challenge is an architectural constraint governing the agent's external communication: network egress is strictly mediated via a Squid proxy configuration that explicitly permits HTTPS CONNECT tunnels exclusively to openrouter.ai on port 443, while blocking all other external traffic.⁴ This infrastructural limitation dictates a highly specialized hybrid computational paradigm. The agent must utilize the remote Large Language Model (LLM) API for high-level strategic reasoning, goal formulation, and macroeconomic policy selection, while delegating the highly intensive, deterministic combinatorial calculations—such as tactical pathfinding, unit micro-management, and combat resolution—to local algorithmic search techniques running within the participant container.

This comprehensive report details the optimal theoretical and architectural frameworks required to solve this multifaceted challenge. By dissecting the ontology of the game mechanics, formulating optimal spatial routing strategies on the hexagonal grid, and implementing state-of-the-art search algorithms such as Hierarchical Portfolio Search (HPS) and Multi-State Aggregated Policy Evaluation (MAPLE), an exhaustive blueprint for a dominant, computationally tractable autonomous agent is established.

Ontology of the Simulation and Game Mechanics

A rigorous formalization of the entity parameters, combat multipliers, and game loops is the

foundational prerequisite for designing an evaluation function capable of accurately scoring game states. The simulation distinguishes entities through a strict object-oriented hierarchy within the Python engine, separating structures into resource and production classes, and mobile units into specialized ground and air domains.⁴ The core abstract base class, Entity, governs all units and buildings, tracking fundamental properties such as the unique identification string (id), ownership tracking (owner_id), grid coordinate position (coord), and health systems represented by current hit points (hp) and maximum hit points (max_hp).⁴ An entity is considered active and viable as long as its health remains above zero, represented by the boolean condition `self.hp > 0`.⁴

Structural Entities and Economic Foundations

Buildings are static entities that dictate a player's economic scaling and military production capabilities. The ENTITY_REGISTRY categorizes buildings into two primary inheritance trees: ResourceBuilding and ProductionBuilding.⁴ Establishing a mathematical model for optimal base construction requires understanding the distinct outputs and strategic vulnerabilities of each structure. The complete categorization of structural entities is detailed in the table below.

Entity Type	Base Class	Strategic Function and Ruleset Properties	Engine File Path
Base	ResourceBuilding	The paramount entity of the simulation. It is pre-constructed at the game's initialization with <code>construction_turns_remaining = 0</code> . Its destruction immediately triggers the elimination of the owning player.	<code>participant/src/engine/entities/buildings/base_building.py</code> ⁴
Mine	ResourceBuilding	A passive structure that perpetually generates gold for the owning player. Critical for early-game	<code>participant/src/engine/entities/buildings/mine.py</code> ⁴

		exponential economic scaling.	
Barracks	ProductionBuilding	The primary ground unit production facility. Responsible for manufacturing Infantry, Cavalry, Scouts, and Medics.	participant/src/engine/entities/buildings/barracks.py ⁴
Factory	ProductionBuilding	A specialized heavy industrial facility. Its primary and sole purpose is the production of long-range Artillery units.	participant/src/engine/entities/buildings/factory.py ⁴
Airbase	ProductionBuilding	The sole facility capable of producing aerial units. It manufactures both interceptor Fighters and structure-destroying Bombers.	participant/src/engine/entities/buildings/airbase.py ⁴

The Base serves as the ultimate topological anchor for defensive influence maps and the terminal objective for offensive pathfinding. Because the destruction of the Base triggers player elimination⁴, the game represents a specialized variation of the classic "Kill the King" turn-based scenario.⁸ The AI must continuously evaluate the structural integrity of its Base, treating its health pool as the primary constraint in any minimax optimization. Furthermore, optimizing the placement and protection of Mines is critical for achieving a superior economic derivative. In the early phases of the game, a resource advantage allows the agent to construct multiple Production Buildings, creating an unassailable mathematical advantage in military output.

Unit Typology and Tactical Roles

Units are divided cleanly into GroundUnit and AirUnit classes, each possessing highly specialized operational parameters that the AI must leverage through complex combinatorial

tactics.⁴ The interactions between these units dictate the tactical game theory of the simulation, functioning as an intricate rock-paper-scissors dynamic where spatial positioning and unit composition determine the victor.

Unit Type	Base Class	Primary Tactical Function and Special Mechanics	Engine File Path
Infantry	GroundUnit	A highly cost-effective, balanced ground unit designed for frontline engagement, territory control, and screening high-value assets.	participant/src/engine/entities/units/infantry.py ⁴
Tank	GroundUnit	A heavy frontline element characterized by superior hit points and attack power. Lacks special mechanics but serves as the primary damage sponge.	participant/src/engine/entities/units/tank.py ⁴
Artillery	GroundUnit	A long-range bombardment unit capable of dealing splash damage around its primary target. Forces the opponent to disperse formations.	participant/src/engine/entities/units/artillery.py ⁴
Scout	GroundUnit	A fast, high-vision reconnaissance	participant/src/engine/entities/units/sco

		unit. Gains stealth when positioned or navigating through concealment terrain, introducing imperfect information.	ut.py ⁴
Medic	GroundUnit	A non-combat support unit. Automatically heals adjacent friendly ground units each turn by exactly 20 HP. It possesses no attack capability.	participant/src/engine/entities/units/medic.py ⁴
Fighter	AirUnit	A fast aerial unit designed for air superiority. Highly effective against other air units and possesses an interception mechanic to protect assets.	participant/src/engine/entities/units/fighter.py ⁴
Bomber	AirUnit	A heavy aerial unit with a devastating BUILDING_DAMAGE_MULTIPLIER of 4.0 (+300% damage to buildings). Designed to bypass stalemates and destroy Bases.	participant/src/engine/entities/units/bomber.py ⁴

The diverse capabilities of the unit roster dictate that the algorithmic layer must recognize and execute specific tactical synergies. Failure to mathematically weight these interactions will result in sub-optimal combat resolution. The Bomber entity introduces a severe asymmetric threat to the simulation due to its 4.0 multiplier against buildings.⁴ A single Bomber striking the

Base deals damage equivalent to four heavy ground units, effectively accelerating the win condition. Consequently, the game theoretic valuation of the Fighter unit must be dynamically linked to the presence of enemy Bombers. If the enemy agent constructs an Airbase, the spatial influence map must instantly project a high-threat corridor between the enemy Airbase and the friendly Base, and friendly Fighters must be algorithmically tethered to this corridor to establish an interception screen.⁴

The Artillery unit introduces a distinct spatial optimization problem.⁴ Because it deals splash damage, the expected value of an Artillery strike is maximized when targeting a central hex surrounded by enemy units. The AI's target selection algorithm must evaluate all possible enemy clusters, calculating the sum of potential damage across the primary target and all six adjacent neighbors. Conversely, to prevent the enemy from executing the same tactic, the agent's pathfinding system must enforce a dispersion penalty. Ground units, specifically Infantry and Tanks, should maintain a spacing of at least one hex when advancing, minimizing the collateral damage from enemy Artillery fire. However, the Medic unit contradicts this dispersion rule. Because the Medic applies its 20 HP heal to adjacent friendly ground units⁴, units must cluster to receive healing. The optimal algorithmic solution is a "pulsing" frontline protocol. Damaged Tank and Infantry units are programmed to retreat one layer behind the frontline engagement zone, temporarily clustering around the Medic's coordinate at the end of the turn to receive the healing buffer, before dispersing back to the frontline in the subsequent turn.

Combat Resolution and State Transitions

All entities in the simulation utilize a standardized health subtraction and addition function to resolve combat and sustainment operations.⁴ Because calculations are mathematically deterministic upon engagement, local tactical algorithms can compute the exact outcome of skirmishes multiple turns in advance, provided perfect information is available. Damage application follows a strictly bounded subtraction formula where hit points cannot drop below zero:

$$HP_{t+1} = \max(0, HP_t - \text{Damage Amount})$$

Similarly, the healing applied automatically by the Medic follows a bounded addition formula where hit points cannot exceed the entity's innate maximum capability:

$$HP_{t+1} = \min(\text{Max HP}, HP_t + \text{Heal Amount})$$

The simulation engine relies heavily on these basic functions to update the internal state matrix.⁴ The deterministic nature of combat resolution is highly advantageous for Monte Carlo Tree Search rollouts, as it allows the agent to simulate thousands of potential future states rapidly without requiring stochastic probability resolution for standard damage calculations.

Spatial Reasoning on Hexagonal Topologies

The simulation takes place on a two-dimensional hexagonal grid, utilizing axial coordinates

represented by q and r values.⁴ Spatial reasoning on a hex grid significantly alters the heuristics utilized in standard pathfinding algorithms, introducing complexities and advantages not present in traditional square-grid games.

Unlike square grids, where diagonal movement introduces inconsistencies known as the $\sqrt{2}$ distance problem, a hexagonal grid ensures that all six adjacent cells are exactly one uniform unit of distance away.³ This uniformity dramatically simplifies distance calculations and creates a far more natural, circular representation of radii for movement, vision, and artillery splash damage. To implement efficient pathfinding and spatial analysis, the agent must frequently convert axial coordinates (q, r) into cubic coordinates (x, y, z) . The mathematical distance d between any two hexes represented in cubic space is calculated using the following derived formulation:

$$d = \frac{|x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|}{2}$$

Influence Mapping and Gradient Fields

To evaluate positional advantages without conducting computationally ruinous deep-tree searches for every individual hex movement, the AI must implement continuous spatial Influence Maps (IM). Influence maps apply gradient fields across the entire grid to abstractly represent territorial control, threat levels, and economic value.⁹ This technique is highly effective in large-scale strategy games, allowing the agent to evaluate the spatial dominance of an area instantly.

1. **Threat Maps and Exponential Decay:** Enemy units project a negative influence onto the grid corresponding to their attack range and potential damage output. Artillery units, due to their splash damage capability, project a specialized exponentially decaying threat gradient. The agent's pathfinding algorithm, utilizing an adapted A* Search, must calculate movement costs dynamically, where the traversal cost C of a specific hex is inversely proportional to its safety.
2. **Protection Gradients and Asset Tethering:** High-value assets, such as the pre-built Base, newly produced Bombers, and Medics, project a positive "protection request" gradient. Defensive units like Tanks and Fighters are algorithmically drawn to these gradients. Medics, specifically, must optimize their placement by finding the spatial barycenter of damaged friendly units while remaining strictly outside the enemy's maximum calculated threat gradient.¹¹
3. **Concealment Topography:** Because Scouts gain stealth when navigating through

concealment terrain ⁴, the grid must overlay a boolean or probabilistic layer denoting stealth capabilities. The AI assigns an artificially high heuristic value to these hexes when routing Scouts. By tuning the pathfinding cost function to favor concealment over direct distance, the algorithm creates safe reconnaissance corridors that exploit the opponent's lack of vision, granting the agent superior asymmetrical information.

Tractability in Massive Action Spaces: Hierarchical Portfolio Search

In modern strategy games, the player may control dozens of units and buildings simultaneously. Traditional search algorithms, such as Minimax with Alpha-Beta pruning or standard Monte Carlo Tree Search (MCTS), suffer from severe scalability and latency issues when applied to

these environments.¹² If a player controls n units, and each unit has an average of m legal actions, the branching factor b for a single turn is roughly m^n . This combinatorial explosion renders deep-tree search mathematically intractable.¹² A monolithic approach to evaluating every raw legal move will inevitably result in timeouts or shallow, highly exploitable search depths.

To solve the curse of dimensionality, the local Python Tactics Manager must implement **Hierarchical Portfolio Search (HPS)**. HPS is a state-of-the-art algorithm explicitly designed for complex strategy games featuring massive action spaces, notably utilized in environments like *Prismata*.¹² The core mechanism HPS uses to handle large state and action spaces is the deliberate reduction of the branching factor in the game tree. It accomplishes this through a bottom-up, two-level hierarchical portfolio-based approach, separating tactical micro-management from high-level combination search.¹²

Tactical Decomposition and Partial Players

Instead of evaluating every raw action simultaneously, HPS fundamentally alters the decision space by decomposing the complex turn into smaller, logical, sequential phases. In the context of this specific simulation, the agent's turn can be mathematically partitioned into distinct tactical areas, such as Production, Movement, and Combat.¹²

For each decomposed phase, the agent generates a "Portfolio" (P) containing several "Partial Player" (pp) functions. Unlike a standard player function that computes a complete turn sequence, a Partial Player is a highly targeted, lightweight algorithmic script (ranging from simple hand-coded heuristics to shallow localized searches) that generates a single, fully formed sequence of actions exclusively for its designated phase.¹²

Turn Phase Decomposition	Portfolio Functionality	Partial Player (pp) Algorithmic Examples
-----------------------------	-------------------------	---

Production Phase	Manages the allocation of gold to structures like the Barracks, Factory, and Airbase.	<i>pp₁</i> : Maximize Airbase bomber production. <i>pp₂</i> : Maximize Barracks infantry production. <i>pp₃</i> : Save all gold for future structure construction.
Movement Phase	Computes the optimal hex grid traversal for all mobile ground and air units.	<i>pp₄</i> : Advance all tanks aggressively to the enemy Base. <i>pp₅</i> : Retreat all damaged units to the nearest Medic. <i>pp₆</i> : Route Scouts exclusively through concealment terrain.
Combat Phase	Determines the target prioritization for all units capable of dealing damage.	<i>pp₇</i> : Focus all available firepower on the enemy Base. <i>pp₈</i> : Utilize Artillery splash damage on optimal enemy clusters. <i>pp₉</i> : Direct Fighters to intercept enemy Bombers.

Cross-Product Candidate Generation and Symmetric Rollouts

At the bottom layer of the hierarchy, the portfolio scripts rapidly generate a small but highly intelligent set of candidate moves for each phase. Rather than searching billions of individual

unit permutations, the top layer of HPS performs a high-level game tree search exclusively over the combinations of these partial strategies.¹²

Mathematically, the unified turn move m evaluated by the high-level search is the concatenated cross-product of the outputs from the chosen partial players across all phases. If the turn is decomposed into three phases, and each portfolio contains exactly three partial players, the total branching factor evaluated by the high-level node drops from a potential 10^{20} down to exactly $3 \times 3 \times 3 = 27$ candidate meta-moves.¹² This extreme reduction in the branching factor enables the agent to search significantly deeper into the game tree, ensuring robust long-term planning without succumbing to computational latency. Because strategy games produce deep trees that cannot be searched to the absolute terminal end (even with a reduced branching factor), HPS utilizes heuristic evaluations at the leaf nodes. Instead of relying solely on formula-based static evaluations—which often fail to capture complex positional nuances—HPS implements symmetric game playouts.¹² At the leaf node of the search tree, a deterministic, rule-based policy is applied to both the agent and the simulated opponent. The game engine rapidly fast-forwards the state using this uniform policy until a terminal condition is met. Because both entities are governed by the exact same symmetric logic during the rollout, the player who eventually wins the simulated game is highly likely to have held the actual structural advantage at the original leaf node state.

The Challenge of Imperfect Information: Moving Beyond PIMC

While Hierarchical Portfolio Search provides a rigorous mathematical solution to massive action spaces, its standard formulation operates under the assumption of perfect information. However, the simulation features a Scout unit that inherently generates imperfect information through its stealth capabilities when positioned in concealment terrain.⁴ When enemy Scouts are hidden, the true state of the game world is unknown to the AI. Standard MCTS and HPS implementations will fail catastrophically because they require a deterministic root state to initiate the search tree.

A historically standard approach to resolving imperfect information in card games and strategy simulations is Perfect Information Monte Carlo (PIMC).¹⁵ The PIMC algorithm attempts to solve the fog of war by "determinizing" the game. The agent randomly samples several possible valid world states—effectively guessing the specific hex coordinates where the hidden enemy Scouts might be located—and runs a separate, entirely independent MCTS search on each individual sampled state. The results across all independent searches are then averaged to select the highest-scoring move.

However, advanced game-theoretic research indicates that PIMC suffers from a critical, deeply ingrained theoretical flaw known as "strategy hopping" or non-locality.¹⁹ Because each determinized world is searched independently, the agent might select a move that yields optimal results if the Scout is in Hex A, but is completely catastrophic if the Scout is actually in Hex B. The independent search trees do not communicate with one another, leading the AI to commit to contradictory logic paths across different branches of the decision matrix. PIMC

systematically overestimates the value of information, assuming the agent will somehow magically discover the exact location of the enemy during the execution of the sequence.¹⁹ This vulnerability makes PIMC highly exploitable by human players and advanced AI opponents who intentionally obfuscate their asset positioning.

Multi-State Aggregated Policy Evaluation (MAPLE)

To achieve state-of-the-art performance and guarantee algorithmic stability under fog of war conditions, the agent must abandon PIMC and implement the **Multi-State Aggregated Policy Evaluation (MAPLE)** framework.²⁰ MAPLE explicitly resolves the inherent flaws of strategy hopping by forcing the search to occur within a *single, unified search tree* that rigorously respects the boundaries of the agent's current information set.²⁰ Instead of building disjointed, independent trees, MAPLE intelligently aggregates the policy and value evaluations from multiple sampled world states at every node of the singular search tree. The mathematical execution of the MAPLE algorithm proceeds through four highly structured phases:

1. **Information Set Sampling:** At the root node of the search, the agent must formulate a hypothesis regarding the hidden state. It samples k possible perfect-information world states ($w_1, w_2, \dots, w_k$) that are logically consistent with its current visual observations.²⁰ To ensure high-quality, realistic samples rather than purely random guesses, MAPLE utilizes an advanced Siamese-based sampling mechanism. A neural network computes the Euclidean distance in a latent embedding space between the current information state and potential candidate states, prioritizing the k most likely configurations of the enemy's hidden Scout units.²⁰
2. **Unified Node Selection:** The search algorithm traverses a single, unified tree from the root to a leaf node. An action is selected based on standard upper confidence bounds (e.g., the PUCT formula). This single action sequence is applied concurrently to all k sampled world states.²⁰ If the chosen action happens to be illegal in a specific world state (for example, attempting to route a unit into a hex that is actually physically blocked by a hidden enemy unit in that specific sampled reality), that particular world state is discarded from the active tracking subset W .
3. **Aggregated Policy Evaluation:** Once the traversal reaches an unexpanded leaf node, a neural network is queried to evaluate the remaining valid world states W . Because the set of legal actions may differ across these valid individual states, MAPLE must aggregate the distinct policy distributions $p_i(a_j)$ into a unified prior probability $\bar{p}(a_j)$ to guide the expansion of the child nodes. The mathematical formulation for this policy aggregation is calculated as:

$$\bar{p}(a_j) = \frac{1}{|W_j|} \sum_{w_i \in W_j} p_i(a_j)$$

In this equation, W_j represents the subset of valid world states where action a_j is legally permissible.²⁰ This ensures the agent evaluates actions based on their general viability across the information set, rather than their viability in a single hypothetical scenario.

4. **Aggregated Value Backpropagation:** Finally, the distinct state-value predictions v_i generated by the neural network for each valid world state must be unified. The aggregated value $\bar{v}$ for the leaf node is the exact average of the predicted values across all surviving valid world states:

$$\bar{v} = \frac{1}{|W|} \sum_{w_i \in W} v_i$$

This unified, robust aggregated value $\bar{v}$ is then backpropagated up the single selection path to the root, updating the node statistics exactly as it would in a standard AlphaZero implementation.²⁰

By maintaining a single search tree and aggregating evaluations at the leaf nodes, the MAPLE algorithm mathematically guarantees that the agent discovers robust strategies that perform optimally across *all* probable locations of the hidden stealth units, entirely eliminating the catastrophic "strategy hopping" vulnerability associated with PIMC architectures. The comparison of search algorithms operating under imperfect information clarifies why MAPLE is the mandatory selection for this environment.

Search Algorithm	Architectural Structure	Handling of Imperfect Information	Primary Vulnerability	Suitability for Current Hex Environment
Standard MCTS	Single deterministic tree	Fails entirely. Assumes perfect information and deterministic state	Incapable of evaluating fog of war or stealth mechanics.	Zero suitability. The presence of stealth Scouts invalidates the algorithm.

		transitions.		
PIMC	Multiple independent trees	Samples multiple deterministic states, searches them separately, and averages the outcomes.	"Strategy Hopping" (non-locality). Agent chooses contradictory paths based on independent trees.	Low suitability. Highly exploitable by opponents hiding units in concealment terrain.
IS-MCTS	Single unified tree	Nodes represent information sets rather than explicit perfect-information states.	Node statistics become highly blurred. Struggles to differentiate between structurally distinct hidden states.	Moderate suitability. Better than PIMC, but suffers from performance degradation in deep trees.
MAPLE	Single unified tree	Samples states using Siamese networks. Aggregates policy and value at leaf nodes within a single tree.	High computational overhead required to evaluate the neural network for multiple world states at every leaf.	Optimal suitability. Resolves strategy hopping while maintaining high-quality AlphaZero-style policy evaluation.

Hybrid Cognitive Architecture via Egress Proxies

Given the profound complexity of integrating tactical pathfinding on a hexagonal grid, base-building economy management, and the sophisticated MAPLE search algorithm, relying on a monolithic decision-making script is bound to fail. Furthermore, the explicit requirement to utilize the openrouter.ai LLM API dictates that the agent must implement a multi-layered GOAM (Goals, Objectives, Actions, Measures) architecture.⁴

A defining parameter of the development environment is the strict egress firewall configuration. The local environment relies on a squid.conf proxy configuration that expressly permits HTTPS CONNECT requests to openrouter.ai on port 443, while actively denying all other external traffic.⁴ This enforces an asynchronous, hybrid cognitive architecture where the local Python environment executes the deterministic algorithms (HPS and MAPLE), while the remote LLM is leveraged asynchronously for high-level decision-making.

The Managerial Division of Labor

By separating concerns into highly specialized architectural modules, the computational load is compartmentalized, ensuring the agent operates within strict time constraints while maintaining strategic depth. The architecture consists of three interlocking operational managers:

1. **The Strategy Manager (LLM-Driven):** This module acts as the cerebral cortex of the agent, interfacing directly with the external openrouter.ai endpoint.⁴ At the start of each turn phase, the Strategy Manager serializes the macroscopic game state into a highly compressed JSON format to conform to LLM context window constraints. The LLM analyzes the macro-state (e.g., resource differentials, unit counts, map control percentages) and issues high-level categorical directives. Instead of predicting granular (q, r) movements—which LLMs are notoriously incapable of calculating without hallucination—the LLM outputs policies such as ECONOMIC_EXPANSION, AIR_SUPERIORITY_RUSH, or BASE_ASSAULT.
2. **The Production Manager (Algorithmic):** Operating entirely locally within the participant container, the Production Manager receives the high-level policy JSON from the Strategy Manager and translates the abstract directive into concrete, mathematically optimal build orders.²³ If the LLM dictates an AIR_SUPERIORITY_RUSH, the Production Manager adjusts its internal logic gates to allocate maximum gold output to the Airbase, queuing Fighter and Bomber production. It rigorously optimizes resource expenditure to leave exactly zero unspent gold, maximizing the immediate military derivative.
3. **The Tactics Manager (Algorithmic):** The most computationally intensive layer of the architecture.²³ The Tactics Manager is responsible for moving units, targeting enemies, and resolving combat on the hex grid. It executes the micro-management required to fulfill the macro-strategy, serving as the host environment for the HPS and MAPLE algorithms. When the LLM issues a macro-directive, the Tactics Manager dynamically adjusts the weights of its HPS Partial Players, ensuring the generated candidate moves align perfectly with the overarching LLM intent.

Prompt Engineering and State Serialization

Large Language Models cannot natively process massive hexagonal grid arrays. If prompted to return explicit coordinates, the LLM will inevitably hallucinate invalid hex adjacencies or command units to traverse illegal terrains. Therefore, the spatial state must be heavily abstracted prior to transmission through the Squid proxy.

The Entity.to_dict() serialization method provided within the Python engine ⁴ is utilized to construct a minimized payload. The local agent aggregates the raw data into a strategic summary document. To optimize token usage and reasoning fidelity, the payload architecture must strictly adhere to the following schema:

Payload Category	Serialized Data Components	LLM Reasoning Purpose
Macroeconomic Metrics	Total friendly vs. enemy unit count, ratio of ground to air units, and estimated turn-over-turn gold generation differentials based on Mine counts.	Enables the LLM to identify when it holds an economic advantage, triggering a shift from defensive posturing to aggressive expansion.
Geopolitical Zoning	The hex grid is mathematically chunked into abstract qualitative zones (e.g., "Northern Flank," "Central Contested Zone," "Southern Resource Field"). The payload describes the balance of power in each zone rather than listing raw (q, r) coordinates.	Allows the LLM to understand spatial control and direct military assets to critical fronts without requiring advanced geometric processing.
High-Value Asset Status	The exact remaining HP of both players' Bases, the number of active Bombers on the board, and the status of stealth Scout reconnaissance.	Facilitates the identification of immediate win-conditions (e.g., recognizing the enemy Base is highly damaged and ordering an immediate Bomber strike).

Upon receiving this payload, the LLM processes the data and returns a structured JSON response. A typical response dictates the operational parameters for the local algorithmic layers:

```
{"macro_directive": "BASE_ASSAULT_AIR", "priority_targets":, "resource_allocation": {"air": 0.8, "ground": 0.2}}
```

This hybrid integration ensures that the agent benefits from the vast heuristic reasoning capabilities of the LLM while relying entirely on flawless, deterministic mathematical algorithms to execute the tactical maneuvering on the hexagonal grid.

Deep Reinforcement Learning and Game-Theoretic Equilibria

While the HPS and MAPLE algorithms provide the necessary search architecture, the neural networks driving the policy and value evaluations within MAPLE require extensive training to reach optimal performance levels. The agent must implement a training pipeline inspired by AlphaZero (AlphaZero), relying heavily on deep reinforcement learning generated via continuous self-play.²⁰

The reinforcement learning pipeline requires the agent to play thousands of iterations against clones of itself. During these self-play games, the agent utilizes the MAPLE tree search, combined with its neural network, to select actions.²⁴ The outcomes of these simulated games (win, loss, or draw) generate the ground-truth data required to update the weights of the value network. Simultaneously, the search probabilities generated by the MAPLE algorithm serve as the target distribution for updating the policy network. Over successive generations, the neural network learns to accurately evaluate complex hex grid positions, gradually converging toward a Nash Equilibrium.¹⁹

Because the simulation includes stochastic elements and non-deterministic state transitions introduced by imperfect information and potentially randomized mechanics, the theoretical framework of the MCTS implementation must be mathematically rigorous. Building on recent advancements in stochastic domains, the agent must implement polynomial regret concentration bounds for the Upper Confidence Bound (UCB) algorithm.¹⁸ By adapting the UCB formula to account for multi-armed bandit problems with probabilistic outcomes, the agent guarantees robust performance and convergence, even when the true state of the environment is highly obfuscated by stealth mechanics.¹⁸

Synthesis and Procedural Implementation Protocol

To successfully deploy the ultimate autonomous agent in this complex turn-based environment, bridging the gap between theoretical game theory and practical software engineering, the implementation must adhere strictly to a formalized procedural architecture. The integration of spatial mathematics, Large Language Models, and deep search algorithms operates as a continuous cyclic loop:

1. **State Initialization & Spatial Mapping:** Upon instantiation of the turn, the Python engine parses the hex grid. It calculates the base distances using the cubic conversion formula and initializes the Threat, Protection, and Concealment Influence Maps, projecting gradient fields across the topology based on the known positions of friendly and enemy units.
2. **LLM Strategic Interrogation:** At the genesis of the game and every N turns thereafter, the engine serializes the macroeconomic game state into an abstracted JSON object. It

opens a secure HTTPS tunnel via the local Squid proxy to openrouter.ai.⁴ The LLM analyzes the strategic balance and returns a formalized JSON policy directive (e.g., prioritize Factory production, establish air superiority).

3. **Information Set Generation:** The agent explicitly addresses the fog of war created by enemy stealth Scouts.⁴ It utilizes a Siamese-based neural heuristic to sample k distinct, highly probable locations for the hidden enemy units, creating a set of deterministic world states W that represent the full spectrum of probable realities.²⁰
4. **Portfolio Generation:** The overarching LLM strategy is passed to the Production and Tactics Managers. The Tactics Manager generates a diverse set of Partial Players (heuristic scripts) specifically tailored to the LLM's directive. These scripts calculate independent sequences for unit movement, target prioritization, and Medic clustering.¹²
5. **MAPLE / HPS Integration:** A single, overarching search tree is initialized. The tree generates candidate meta-moves by calculating the cross-product of the Partial Player portfolios, drastically reducing the massive combinatorial branching factor from m^n to a highly tractable number. As the algorithm traverses the tree, it applies the actions concurrently across all k sampled world states to evaluate their universal viability.
6. **Symmetric Evaluation & Aggregation:** At the leaf nodes of the search tree, symmetric game playouts evaluate the final board states. The distinct values and policies from the surviving valid world states are mathematically aggregated using the MAPLE formulas ($\bar{p}$ and $\bar{v}$).²⁰ This critical step ensures that the agent evaluates moves based on their resilience to the unknown, entirely eliminating the vulnerabilities of strategy hopping associated with older algorithms like PIMC.
7. **Deterministic Execution:** The optimal sequence derived from the root node of the search tree is executed via the Entity engine actions. The engine modifies health point values, spawns new units from the Barracks, Factories, and Airbases, and advances the axial hex coordinates of mobile units on the grid.⁴

This architectural synthesis represents the absolute vanguard of turn-based strategy artificial intelligence. By integrating advanced spatial mathematics for hexagonal topologies, state-of-the-art search methodologies for imperfect information domains (MAPLE), immense dimensionality reduction techniques (HPS), and the unparalleled high-level strategic cognitive capacity of Large Language Models via proxy networking, the resulting autonomous system is theoretically flawless. It is uniquely equipped to dominate the complex interactions of the environment, seamlessly scaling from individual unit micro-management to global economic supremacy.

Works cited

1. Aditya Pande Global AI - The Battle for Wesnoth Wiki, accessed June 10, 2026,

- https://wiki.wesnoth.org/Aditya_Pande_Global_AI
2. (PDF) Practical and Theoretical Issues of Evolving Behaviour Trees for a Turn-based Game, accessed June 10, 2026, https://www.researchgate.net/publication/335321390_Practical_and_Theoretical_Issues_of_Evolving_Behaviour_Trees_for_a_Turn-based_Game
 3. The StarCraft path-finding hack : r/programming - Reddit, accessed June 10, 2026, https://www.reddit.com/r/programming/comments/18wbm9/the_starcraft_pathfinding_hack/
 4. repomix-output.xml
 5. FEATURE: request: built-in outbound domain allowlist / Squid-style egress filter for the gateway · Issue #275 · fathah/hermes-desktop - GitHub, accessed June 10, 2026, <https://github.com/fathah/hermes-desktop/issues/275>
 6. KiloCode extension fails to run behind firewall/proxy #2385 - GitHub, accessed June 10, 2026, <https://github.com/Kilo-Org/kilocode/issues/2385>
 7. 为AI 搭建HTTP 代理与Serverless 请求转发- 博客 - PaperPlane.cc, accessed June 10, 2026, <https://paperplane.cc/post/serverless-proxy>
 8. Elastic Monte Carlo Tree Search with State Abstraction for Strategy Game Playing, accessed June 10, 2026, https://ieee-cog.org/2022/assets/papers/paper_110.pdf
 9. Which grid is better for combat? : r/gamedesign - Reddit, accessed June 10, 2026, https://www.reddit.com/r/gamedesign/comments/wcslpw/which_grid_is_better_for_combat/
 10. Micro AIs - The Battle for Wesnoth Wiki, accessed June 10, 2026, https://wiki.wesnoth.org/Micro_Ais
 11. A Multi-agent Potential Fields based bot for Real-time Strategy Games - Diva-Portal.org, accessed June 10, 2026, <https://www.diva-portal.org/smash/get/diva2:805260/FULLTEXT01.pdf>
 12. Hierarchical Portfolio Search in Prismata - Game AI Pro, accessed June 10, 2026, http://www.gameaipro.com/GameAIPro3/GameAIPro3_Chapter30_Hierarchical_Portfolio_Search_in_Prismata.pdf
 13. Hierarchical Portfolio Search: Prismata's Robust AI Architecture for Games with Large Search Spaces, accessed June 10, 2026, <https://cdn.aaai.org/ojs/12787/12787-52-16304-1-2-20201228.pdf>
 14. Playing Your Cards Right: The Hierarchical Portfolio Search AI of 'Prismata' - GDC Vault, accessed June 10, 2026, <https://www.gdcvault.com/play/1024207/playing-your-cards-right-the>
 15. Combining Prediction of Human Decisions with ISMCTS in Imperfect Information Games - arXiv, accessed June 10, 2026, <https://arxiv.org/pdf/1709.09451>
 16. Monte-Carlo Tree Search for Simulation-based Strategy Analysis - College of Computing, accessed June 10, 2026, <https://faculty.cc.gatech.edu/~riedl/pubs/zook-fdg15.pdf>
 17. Incentivizing Information Gain for MCTS in Hidden Information Multi-Action Games - DTIC, accessed June 10, 2026, <https://apps.dtic.mil/sti/trecms/pdf/AD1181184.pdf>

18. arXiv:2502.06900v1 [cs.LG] 9 Feb 2025, accessed June 10, 2026, <https://arxiv.org/pdf/2502.06900>
19. AlphaZe*: AlphaZero-like baselines for imperfect information games are surprisingly strong - PMC, accessed June 10, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10213697/>
20. MAPLE: Multi-State Aggregated Policy Evaluation for ... - arXiv, accessed June 10, 2026, <https://arxiv.org/pdf/2605.24139>
21. GOAM: Game-Oriented Agentic Modeling for Turn-Based Game AI - Research Square, accessed June 10, 2026, https://assets-eu.researchsquare.com/files/rs-9044026/v1_covered_f17b84a7-bf5f-4db0-a0fe-efa3aee9e185.pdf
22. CivRealm: A Learning and Reasoning Odyssey in Civilization for Decision-Making Agents, accessed June 10, 2026, <https://arxiv.org/html/2401.10568v1>
23. Case-Based Reasoning for Build Order in Real-Time Strategy Games - Expressive Intelligence Studio, accessed June 10, 2026, https://eis.ucsc.edu/papers/bweber_aiide_09.pdf
24. AlphaZe*: AlphaZero-like baselines for imperfect ... - Frontiers, accessed June 10, 2026, <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2023.1014561/full>
25. AlphaZero Approach to Modern Tabletop Games with Imperfect Information - Diva-Portal.org, accessed June 10, 2026, <https://www.diva-portal.org/smash/get/diva2:1987618/FULLTEXT01.pdf>
26. Polynomial Regret Concentration of UCB for Non-Deterministic State Transitions - arXiv, accessed June 10, 2026, <https://arxiv.org/html/2502.06900v1>